MEKDELA-AMBA UNIVERSITY

Architectural Document

Version: 1.0

Project Title: Online Assignment and Project Submission and Evaluation System

Autours : GROUP 1

No	*GROUP NAME*	*ID*
1	BEWKET AFEWORK	MAU1600290
2	MAHLET TADELE	MAU1600789
3	SAMSON YIRGAW	MAU1601044
4	YARED GIDEY	MAU1601295

Submitted To: Destaye.A

Department : SOFTWARE ENGINNERING

Date: May 9, 2026

Approval Sheet

This document is submitted in partial fulfillment for the course Software architecture and design(Course Code: SEng5232).

Approved by:

Name	***Role***	***Signature***	***Date***
Adisu A.	Project Advisor	_____	//____
Mr. Destaye.A	Course Instructor	_____	//____

Acknowledgements

We would like to express our sincere gratitude to our advisor **Mr. Adisu A.** for his continuous guidance, support, and valuable feedback throughout the development of this project. His expertise and encouragement were instrumental in shaping our work.

We also extend our thanks to our course instructor **Mrs. Destaye A.** for providing us with the foundational knowledge in Software architecture and design and for his insightful reviews of our documentation.

Our appreciation goes to the **College of Computing and Informatics Science** at MAU for providing the resources and environment necessary for this academic endeavor.

Finally, we thank our families and friends for their patience and support during this project.

Table of Contents

Titles **Pages**

Approval Sheet i

Acknowledgements ii

Table of Contents iii

List of Tables v

List of Figures vi

Abstract vii

Abbreviations 4

1. INTRODUCTION

1.1. Purpose 1

.2. Scope.....1

1.3. Overview.....2

1.4.Intended Users.....2

1.5. Reference Material 2

1.6. Definitions and Acronyms 2

2. SYSTEM OVERVIEW AND CONSTRAINTS 4

2.1. System Overview 4

2.2.Constraints 4

3. QUALITY ATTRIBUTE SCENARIOS AND TACTIC SELECTION……5

3.1. Quality Attribute Scenarios …….. ……..5

3.2. Tactic Selection ……10

4. SYSTEM ARCHITECTURE ……..…….10

4.1 Architectural Design ……10

4.2. Module View ……12

4.3. Component and Connector View …….13

4.4. Allocation View ……15

4.5 Design Rationale ……16

5. DATA DESIGN ……..17

5.1 Data Description …….18

5.2 Data Dictionary ……..18

6. COMPONENT DESIGN ……...20

6.1 Use Case Diagram & Use Case Descriptions…….20

6.2 Deployment Diagram…….23

6.3 Environment Description……...24

7. REQUIREMENTS MATRIX ……...26

LIST OF TABLES

Table **Pages**

Table 1.1 Acronyms……..3

Table 3.1 Tactic selection…….10

Table 5.1: Data Dictionary……19

Table 7.1: Requirements Traceability Matrix……26

Table 7.2: Design Decisions ……..27

LIST OF FIGURES

Figure **Pages**

Figure 3.1 Performance Scenario – File Upload During Peak Load ……6

Figure 3.2 Security Scenario – Unauthorized Access Attempt ……...6

Figure 3.3 Availability Scenario – Database Node Failure ……..7

Figure 3.4 Modifiability Scenario – Adding New Rubric Type ……..……..8

Figure 3.5 Usability Scenario – First-Time Student Submission ……8

Figure 3.6 Testability Scenario – Unit Testing a New Component ……..9

Figure 3.7 Reliability Scenario – Handling Unexpected Input ……...……..9

Figure 4.1: Major Subsystems & Data Repositories Interconnection Diagram……12

Figure 4.2: OO Module Decomposition & Class Structure ……13

Figure 4.3: Runtime Component & Connector View (with Sequence Diagram) ……14

Figure 4.4: High-Level Architecture Diagram ……15

Figure 4.5: Component Diagram ……..15

Figure 4.6: Subsystem Interaction Diagram ……16

Figure 4.7: Allocation & Deployment View ……17

Figure 5.1: Database Schema and Entity-Relationship Diagram (ERD).....……..19

Figure 5.2: Data Flow Diagram (Level 0) ……20

Figure 6.1: UML Use Case Diagram ……22

Figure 6.2: Deployment Diagram ……..24

Abstract

The Online Assignment and Project Submission and Evaluation System is a Java Swing-based desktop application architected to modernize and secure academic submission and grading workflows at Mekdela Amba University. Designed using a layered architectural pattern integrated with Model-View-Controller (MVC) and Client-Server paradigms, the system enforces strict separation of concerns across presentation, business logic, data access, and infrastructure layers.

The architecture is driven by prioritized quality attribute scenarios, employing targeted tactics such as role-based access control for security, interface-driven module decomposition for modifiability, asynchronous background processing for performance, and progressive UI disclosure for usability. Guided by foundational design principles including information hiding, dependency inversion, and high cohesion, the system is decomposed into cohesive subsystems for submission management, rubric-based evaluation, notification dispatch, and institutional reporting.

Backed by a normalized postgressSQL relational database, the data design ensures transactional integrity, referential consistency, and centralized auditability. Development follows an architecture-centric iterative approach, with structural decisions validated against component-connector interactions, allocation mappings, and standardized architectural documentation aligned with IEEE 1471 and SAD view templates. By replacing fragmented, paper-based, and email-dependent academic processes with a structured, secure, and architecturally sound digital platform, the system enhances instructional efficiency, reduces feedback latency, and establishes a scalable, maintainable foundation for future academic workflow evolution.

Chapter 1

INTRODUCTION

1.1 Purpose

This Software Architecture Document describes the architecture, structural design, component interactions, data management strategies, and deployment topology of the Online Assignment and Project Submission and Evaluation System. It serves as the authoritative technical blueprint for developers, quality assurance engineers, academic administrators, and system stakeholders, ensuring alignment on architectural decisions, quality attribute realization, and implementation constraints.

1.2 Scope

The Online Assignment and Project Submission and Evaluation System is a desktop-based software application designed to support and manage academic submission and grading activities within Mekdela Amba University. The system provides a centralized platform where students can upload assignments and projects, instructors can evaluate submissions using structured rubrics, and administrators can monitor and manage academic workflows.

The scope of the system includes secure file submission, grading, feedback provision, and record management. It aims to replace traditional paper-based and email-based submission processes with a digital, efficient, and reliable system.

The main goal of the project is to modernize academic workflows by providing a structured and user-friendly platform.

The objectives include improving submission efficiency, ensuring data integrity, enabling fair and consistent grading, and supporting multiple users during peak academic periods. The system benefits include reduced manual work, improved feedback speed, enhanced data security, and better organization of academic records.

1.3 Overview

This document follows a structured architectural documentation approach aligned with IEEE 1471 and the view-based modeling practices outlined in university coursework. Chapter 1 introduces purpose and scope. Chapter 2 outlines system functionality and operational constraints. Chapter 3 formalizes quality attribute scenarios and architectural tactics. Chapter 4 presents module, component-and-connector, and allocation views alongside design rationale. Chapter 5 details data modeling and storage strategies. Chapter 6 specifies component behavior, use cases, deployment topology, and core algorithms. Chapter 7 maps architectural elements to functional requirements.

1.4 Intended Users

This document targets backend and frontend Java developers, desktop UI engineers, database administrators, QA testers, university IT infrastructure personnel, academic coordinators, project sponsors and end users such as teachers/lectures and students.

1.5 Reference Material

- IEEE Std 1471-2000: Recommended Practice for Architectural Description of Software-Intensive Systems.
- Bass, Clements, & Kazman: *Software Architecture in Practice.*
- Course Materials: Introduction to Software Design, Envisioning Architecture, Quality Attributes Architecture, Documenting Architecture.
- Oracle Java Swing Developer Guide & Best Practices.
- Mekdela Amba University IT Security & Data Governance Policy.

1.6 Definitions and Acronyms

Definitions of Terms:

➔ **Software Design Document (SDD)**: A formal document that describes the architecture, components, interfaces, and data of a system.

➔ **Model-View-Controller (MVC)**: An architectural pattern that separates an application into three components: model (data), view (interface), and controller (logic).

- ➔ **User Interface (UI)**: The part of the system through which users interact with the application.
- ➔ **Role-Based Access Control (RBAC)**: A security mechanism that restricts system access based on user roles.
- ➔ **Application Programming Interface (API)**: A set of rules and protocols that allow different software components to communicate.
- ➔ **Java Database Connectivity (JDBC):** A Java API used to connect and execute queries with databases.
- ➔ **Rubric**: A structured scoring guide used to evaluate assignments or projects.
- ➔ **Submission System:** The platform used for uploading, managing, and evaluating academic work.

Acronym

Table 1.1 Acronyms

Acronym	Definition
SAD	Software Architecture Document
SRS	Software Requirements Specification
ASR	Architecturally Significant Requirement
QA	Quality Attribute
RBAC	Role-Based Access Control
C&C	Component and Connector
ERD	Entity-Relationship Diagram
MVC	Model-View-Controller
UI	User Interface (Java Swing)

Abbreviations:

Std	Standard
Doc	Document
App	Application
DB	Database

Chapter 2

SYSTEM OVERVIEW AND CONSTRAINTS

2.1 System Overview

The system orchestrates the complete lifecycle of academic submissions through a structured, secure, and auditable workflow. Students authenticate via institutional credentials, access assigned materials, submit documents or code archives, and monitor grading progress through a desktop Java Swing client. Instructors configure assignment parameters, attach weighted rubrics, evaluate submissions using integrated document viewers, assign scores, and publish structured feedback.

The platform dispatches automated notifications for deadline warnings, submission receipts, grading completion, and appeal windows. Administrators oversee academic calendars, enrollment synchronization, user provisioning, and institutional analytics. Subsystems are decoupled by responsibility to enforce separation of concerns, facilitate parallel development within a four-person team, and enable targeted scaling during high-traffic evaluation periods.

2.2 Constraints

- ★ **Team Structure & Schedule**: A 4-member engineering team operates within a 5-month delivery window, necessitating strict scope management and modular parallel development.
- ★ **Development Stack**: Java SE 17+ is mandated for all layers. Java Swing is required for the desktop client UI. Spring Boot is used for the backend REST API to ensure rapid development and maintainability.

★ **Infrastructure & Network**: Deployment targets the university's on-premise virtualized servers. The desktop application must operate efficiently over campus LAN/WiFi with intermittent bandwidth during peak submission periods.

★ **Security & Compliance**: All academic records must adhere to university data protection regulations. Submissions and grades require encryption at rest and in transit, with strict RBAC enforcement.

★ **Integration**: Synchronization with the existing Student Information System (SIS) is mandatory via secure REST APIs for enrollment and course roster data.

★ **Legacy Migration**: Historical grading data stored in CSV/Excel formats must be migrated into the relational schema without data loss.

Chapter 3

QUALITY ATTRIBUTE SCENARIOS AND TACTIC SELECTION

3.1 Quality Attribute Scenarios

Based on the six-part scenario (Source, Stimulus, Environment, Artifact, Response, Response Measure), the following seven high-priority quality attribute scenarios are defined for the Online Assignment and Project Submission and Evaluation System.

QA-1: Performance Scenario – File Upload During Peak Load

- A student using the Java Swing client uploads a 150MB project archive 10 minutes before the deadline while 300 concurrent users are active on campus LAN; the File Processing Service chunks the file into 10MB segments, uploads asynchronously with progress feedback, validates checksums server-side, stores metadata in PostgreSQL, and returns a submission receipt; upload completes within 10 seconds with 0% data loss, receipt displayed within 1 second, and Swing UI remains responsive without blocking the Event Dispatch Thread.

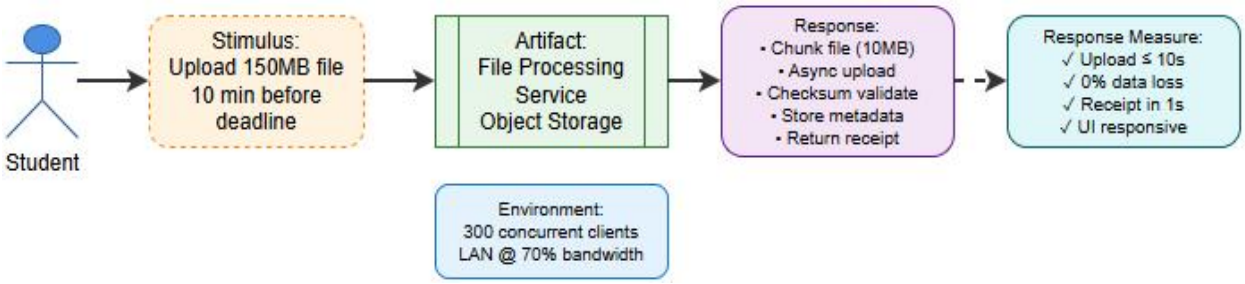


Figure 3.1 Performance Scenario – File Upload During Peak Load

QA-2: Security Scenario – Unauthorized Access Attempt

- An unauthorized external actor or compromised Swing client sends a request with a forged JWT token or manipulated submission ID to access another student's file while security monitoring and audit logging are active; the Authentication Middleware validates the token signature and claims, the Authorization Service checks RBAC permissions against the requested resource, denies the request with HTTP 403, logs the attempt with IP/user-agent/timestamp, and triggers an alert if repeated attempts are detected; 100% of unauthorized attempts are blocked, audit log entries are created within 500ms, and alerts are triggered after 3 failed attempts from the same source within 5 minutes.

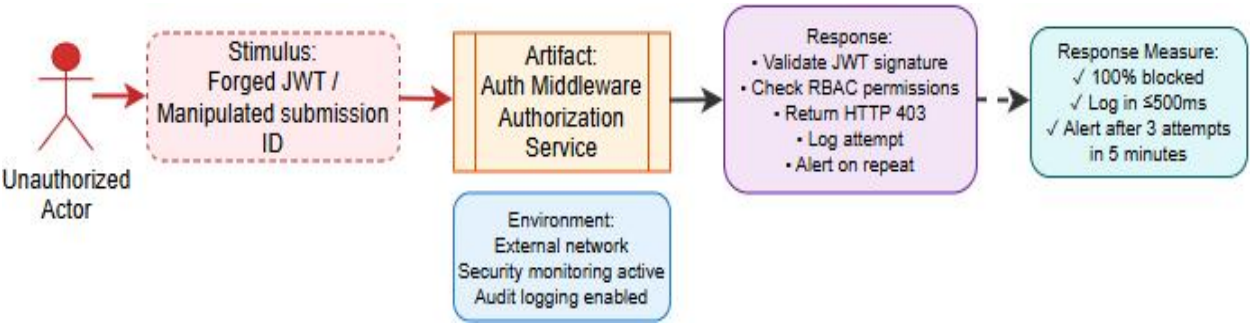


Figure 3.2 Security Scenario – Unauthorized Access Attempt

QA-3: Availability Scenario – Database Node Failure

- The infrastructure monitoring system detects that the primary PostgreSQL node becomes unresponsive due to hardware failure or network partition during an active grading period while high availability mode with active-passive replication and connection pool failover

is enabled; the heartbeat monitor detects the failure within 2 seconds, the connection pool marks the primary as unhealthy and routes new connections to the replica, in-flight transactions are retried or rolled back per ACID policy, and the administrator is notified via alert; failover completes within 8 seconds with zero data loss (replication lag <1 second), no user-visible transaction failures occur, and the alert is delivered to the admin within 3 seconds.

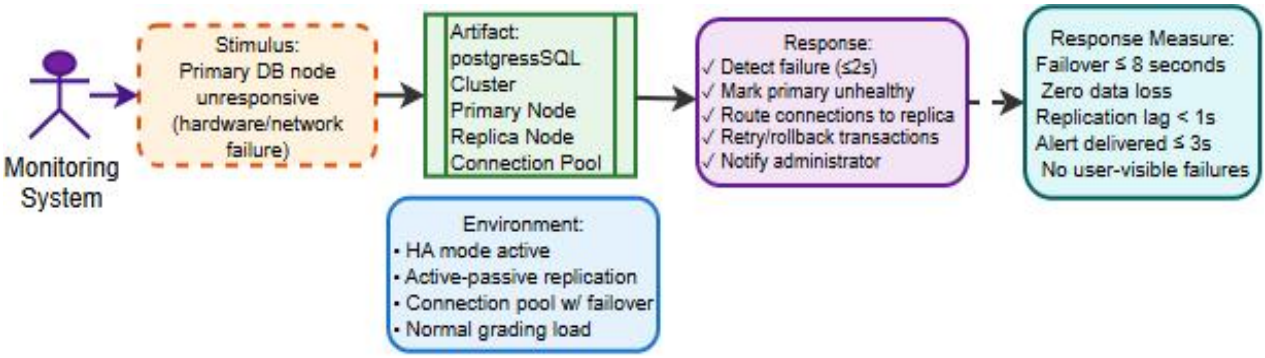


Figure 3.3 Availability Scenario – Database Node Failure

QA-4: Modifiability Scenario – Adding New Rubric Type

- An instructor or academic administrator requests a new custom rubric template via configuration file update or admin UI action while the system operates normally with hot-reload configuration enabled and no code deployment required; the Grading Configuration Service parses the new rubric schema, validates structural integrity and business rules, registers the new type in the runtime registry, updates UI component mappings, and broadcasts the change to connected Swing clients via WebSocket; the new rubric type becomes available for use within 5 minutes of configuration change with zero service interruption, the UI reflects the new type without client restart, and validation errors are reported with clear messages.

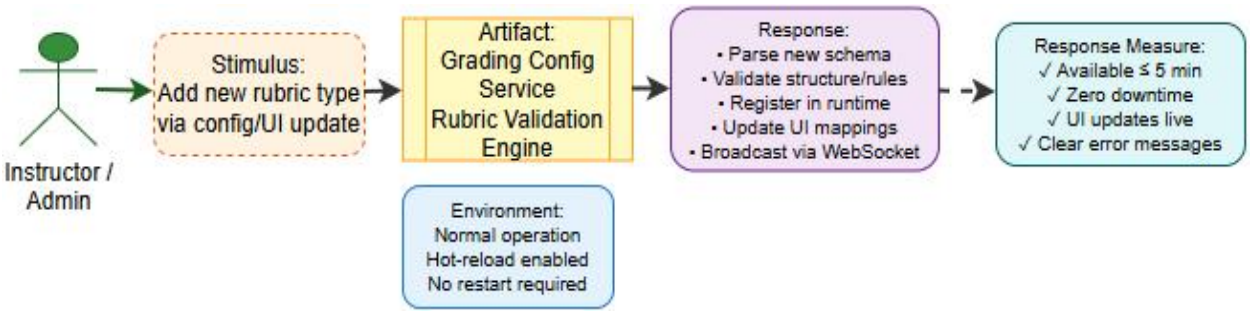


Figure 3.4 Modifiability Scenario – Adding New Rubric Type

QA-5: Usability Scenario – First-Time Student Submission

- A first-time student user launches the Java Swing client and clicks "Submit Assignment" without prior experience while connected via standard campus WiFi on a university lab workstation with onboarding/tutorial mode enabled; the Presentation Layer displays a guided onboarding overlay with tooltips, highlights the drag-and-drop zone with animation, clearly shows file type/size constraints, updates a progress bar during upload, and displays a confirmation dialog with submission ID upon success; the first successful submission is completed within 3 minutes, zero user errors require help desk intervention, and 95% of new users rate UI clarity ≥4/5 in the post-submission survey.

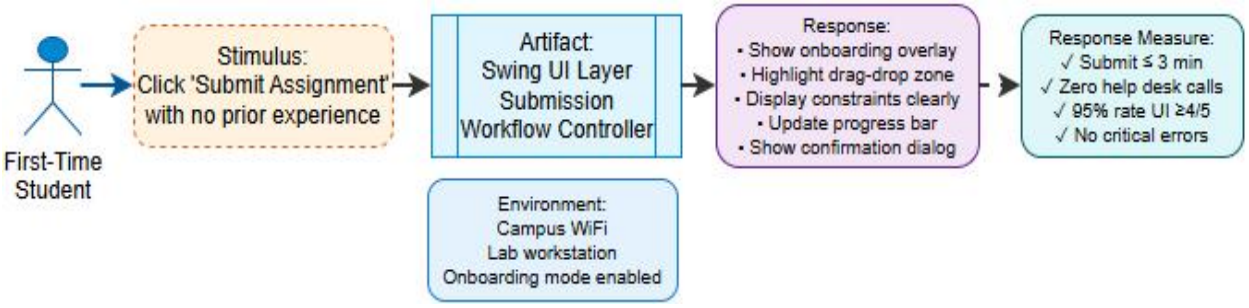


Figure 3.5 Usability Scenario – First-Time Student Submission

QA-6: Testability Scenario – Unit Testing a New Component

- A unit tester performs automated testing on a newly developed SubmissionService component that provides interfaces for controlling behavior and observing output while the system is in a test environment with mock dependencies and logging enabled; the Testability Framework injects mock repositories, executes parameterized test cases

covering boundary conditions and error paths, captures code coverage metrics, and generates a detailed test report; 85% path coverage is achieved within 3 hours, all critical paths are exercised, test execution time remains under 30 seconds per test class, and failures produce actionable diagnostic output.

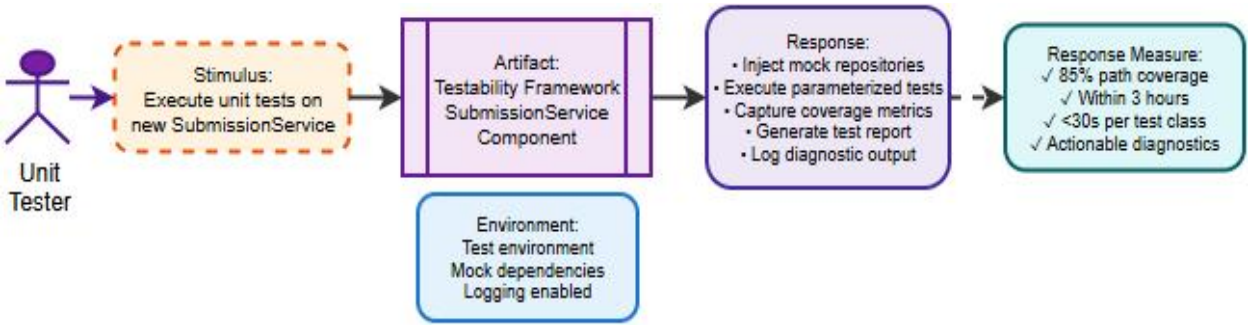


Figure 3.6 Testability Scenario – Unit Testing a New Component

QA-7: Reliability Scenario – Handling Unexpected Input

- A student submits an assignment file with unexpected format or corrupted content through the Swing client while the system operates under normal load with input validation and error handling enabled; the Submission Engine validates file integrity using checksums and format parsers, catches exceptions via structured error handlers, logs the incident with context metadata, returns a user-friendly error message with recovery guidance, and optionally triggers a retry mechanism; the system recovers gracefully without crashing, error messages are displayed within 2 seconds, 99.9% of malformed inputs are handled without data loss, and audit logs capture all recovery actions for traceability.

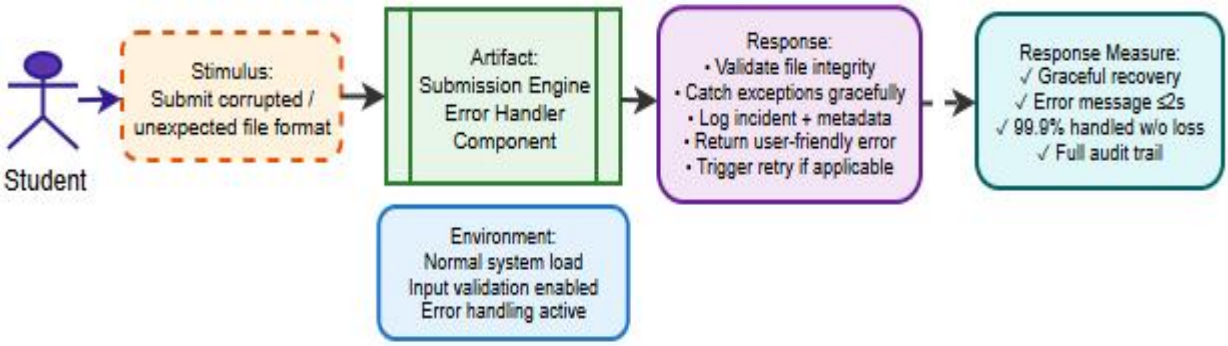


Figure 3.7 Reliability Scenario – Handling Unexpected Input

3.2 Tactic Selection

Table 3.1 Tactic selection

Quality Attribute	Design Concern	Tactic Selected
QA-1 (Performance)	High throughput file handling	Asynchronous processing, resource pooling, chunked storage
QA-2 (Security)	Unauthorized data access	RBAC, JWT validation, HTTPS enforcement, audit logging
QA-3 (Availability)	Single point of failure	Active-passive redundancy, heartbeat monitoring, automatic failover
QA-4 (Modifiability)	Rigid grading logic	Information hiding, reduce coupling, configuration-driven validation
QA-5 (Usability)	Steep learning curve (Desktop UI)	Progressive disclosure, consistent Swing component styling, error prevention dialogs
QA-6 (Testability)	Difficulty verifying component behavior	Dependency injection for mockable interfaces, Parameterized test support,Code coverage instrumentation, Structured logging for diagnostics
QA-7 (Reliability)	System instability from unexpected inputs	Input validation at boundaries,Structured exception handling, Graceful degradation patterns,Comprehensive audit logging

Chapter 4

SYSTEM ARCHITECTURE

4.1 Architectural Design

The system is decomposed into five major subsystems to ensure separation of concerns, parallel development, and scalable maintenance:

- **Client Presentation Subsystem (Java Swing Desktop App)**: Handles all user interactions, form validations, local file caching, and UI state management using the MVC pattern. Communicates with the backend via secure REST APIs.
- **API Gateway & Routing Subsystem (Nginx + Spring Cloud Gateway)**: Acts as the single entry point. Manages TLS termination, request routing, load balancing, rate limiting, and initial CORS validation.

- **Business Logic Subsystem (Spring Boot Services):** Contains core domain services (AuthService, SubmissionService, GradingService, NotificationService). Enforces business rules, transaction management, and role-based access control.
- **Data Access & Persistence Subsystem (JPA/Hibernate Repositories)**: Abstracts database interactions. Provides CRUD operations, query optimization, and transaction boundaries for PostgreSQL and Redis.
- **External Integration & Storage Subsystem**: Manages asynchronous file uploads to MinIO object storage, email/SMS dispatch via SMTP gateways, and synchronization with the University SIS via secure APIs.

Subsystem Collaboration: Clients initiate requests through the Gateway, which routes them to the appropriate Business Service. Services validate inputs, execute domain logic, and delegate persistence to the Data Access layer. File-heavy operations are offloaded to the Storage subsystem asynchronously. Notifications are queued and dispatched independently to prevent UI blocking.

Architecture Pattern: The system utilizes a **Layered Architecture** combined with **Client-Server** communication and **MVC (Model-View-Controller)** at the presentation tier. This pattern was chosen to align with the 4-member team's Java expertise, simplify testing, and enable clear boundary enforcement between UI, business logic, and data persistence.

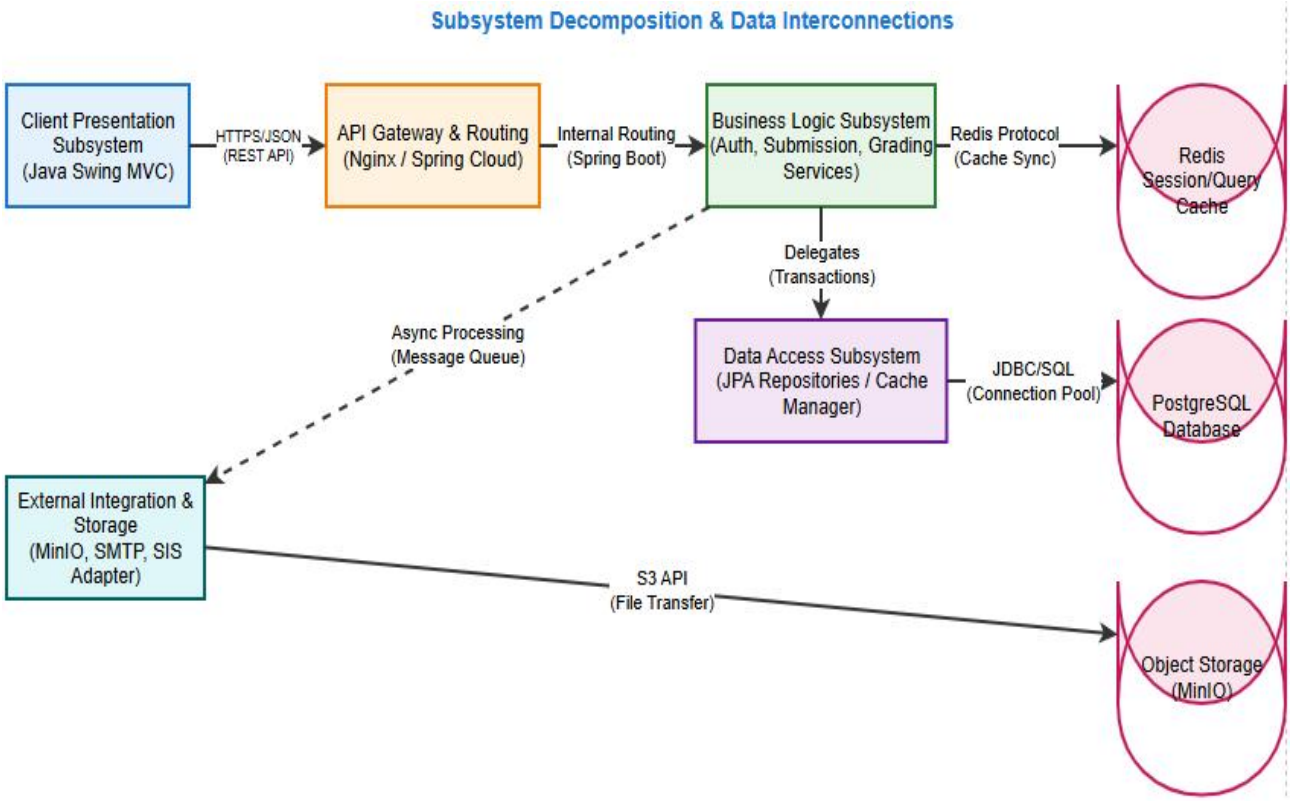


Figure 4.1: Major Subsystems & Data Repositories Interconnection Diagram

4.2 Module View

The system is described using an Object-Oriented decomposition approach. The architecture is structured into cohesive Java packages that map directly to functional responsibilities. Each module exposes a well-defined interface while encapsulating implementation details, adhering to information hiding principles.

- **com.maau.ui (Presentation)**: Contains JFrame windows, JPanel layouts, Swing event listeners, and DTO converters for UI rendering.
- **com.maau.controller (Boundary)**: REST endpoints annotated with @RestController. Validates incoming requests and forwards to services.
- **com.maau.service (Control)**: Implements business logic, transaction demarcation (@Transactional), and rule validation.
- **com.maau.repository (Data Access)**: JPA repositories extending JpaRepository. Handles query execution and pagination.

- **com.maau.domain (Entity)**: JPA entity classes representing database tables with relationships mapped via @OneToMany, @ManyToOne.
- **com.maau.config (Infrastructure)**: Spring configuration classes for security, database pooling, and async executors.

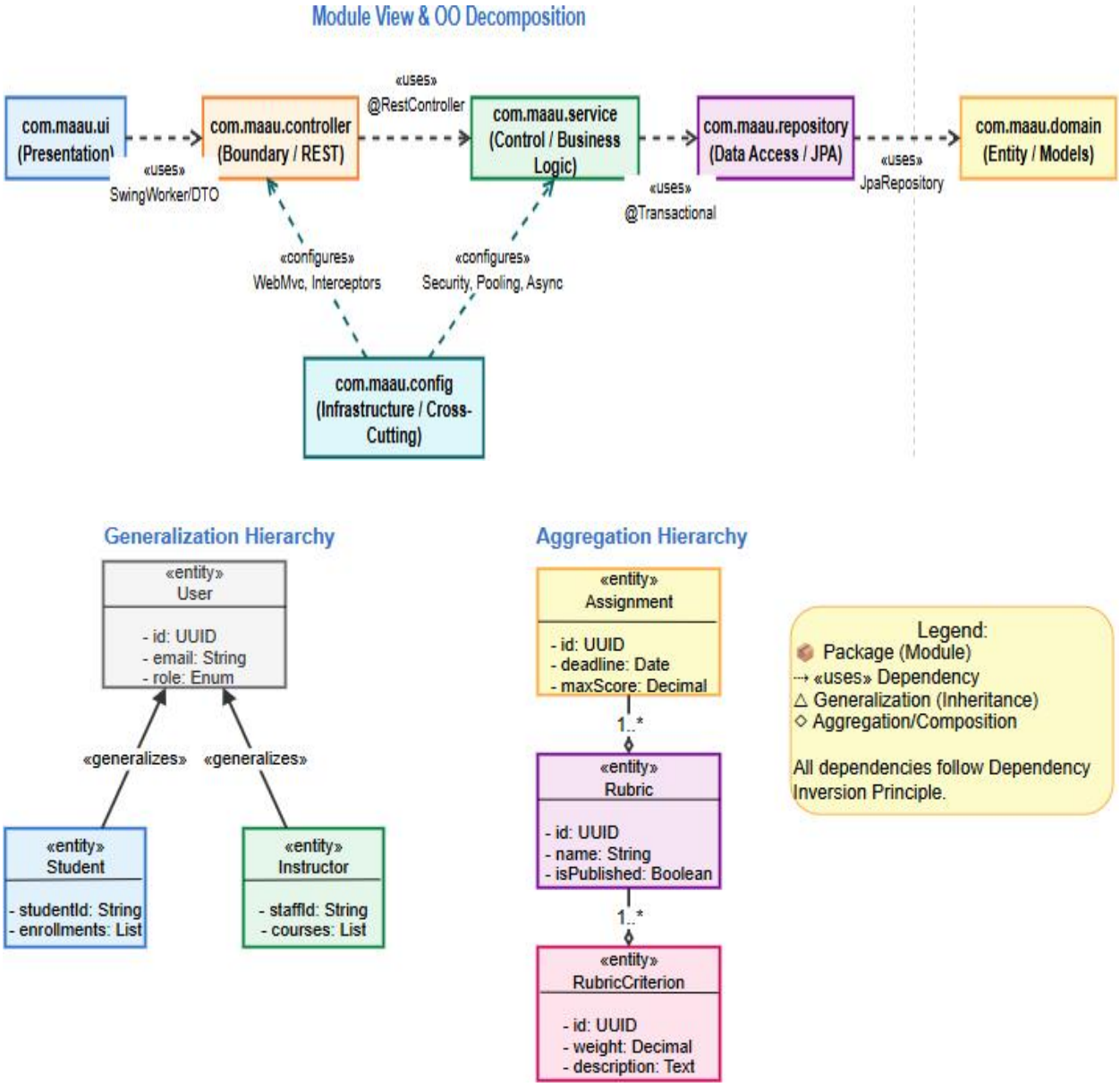


Figure 4.2: OO Module Decomposition & Class Structure

4.3 Component and Connector View

This view describes the runtime architecture, focusing on executing components and their communication protocols.

- ➢ **Executing Components**: SwingClientApp, NginxProxy, AuthService, SubmissionEngine, GradingProcessor, NotificationWorker, PostgreSQLCluster, MinIOStorage.
- ➢ **Connectors**: HTTPS/TLS 1.3 for client-to-server, JDBC Connection Pool for service-to-database, S3 REST API for storage access, WebSocket for real-time grading status, SMTP for email dispatch.
- ➢ **Concurrency Model**: Swing EDT for UI responsiveness, Spring @Async thread pools for file processing, PostgreSQL connection pooling (HikariCP) for DB efficiency.

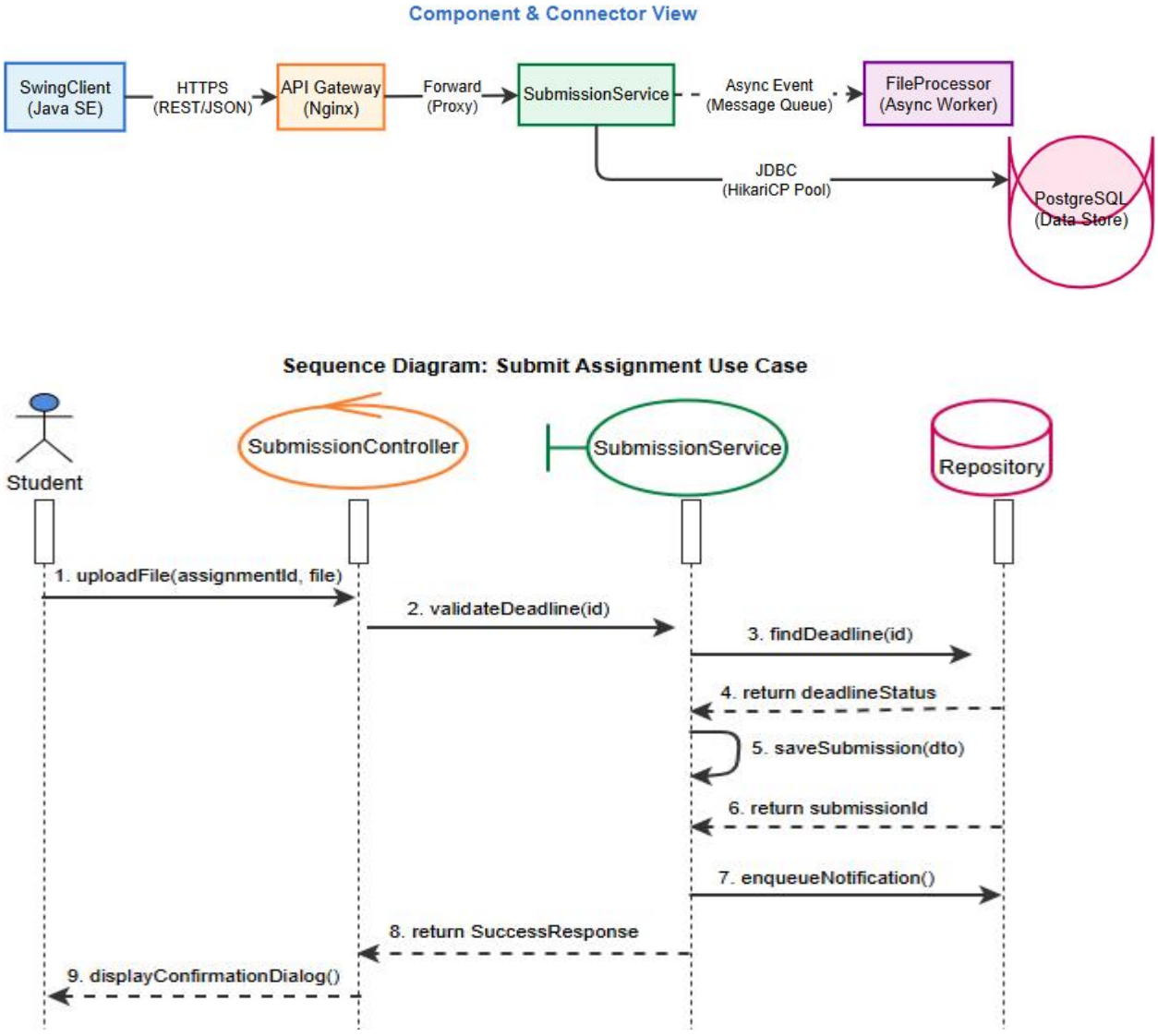


Figure 4.3: Runtime Component & Connector View (with Sequence Diagram)

4.4 Architecture Diagrams

The following diagrams illustrate the system from multiple stakeholder perspectives as required.

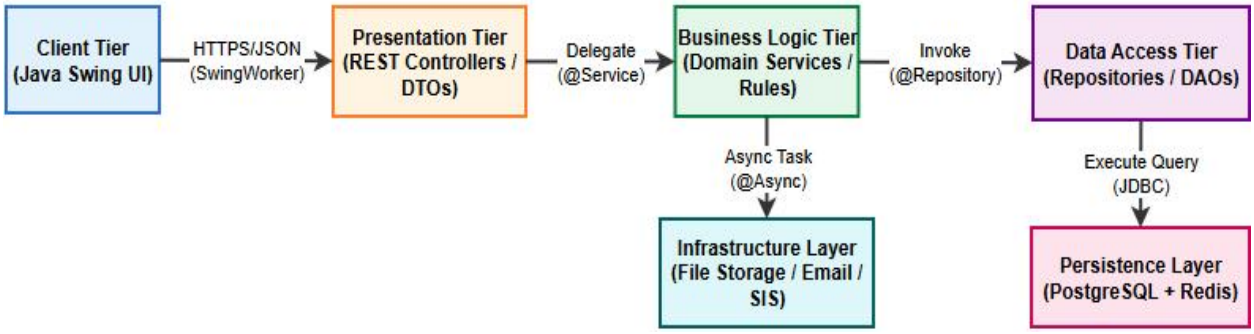


Figure 4.4: High-Level Architecture Diagram

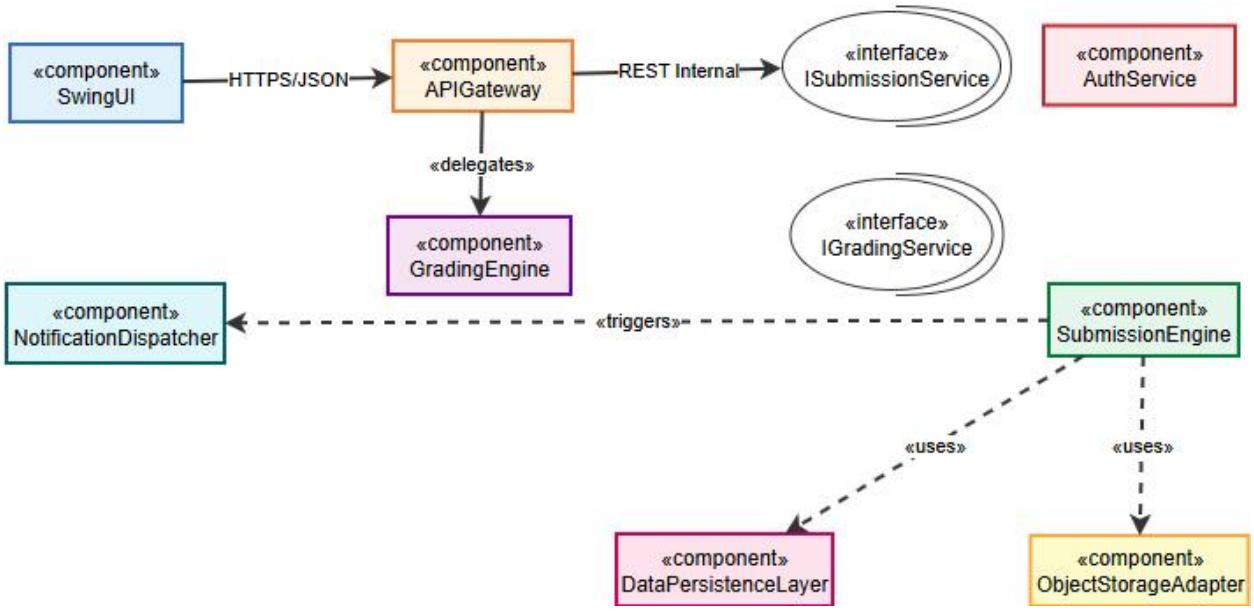


Figure 4.5: Component Diagram

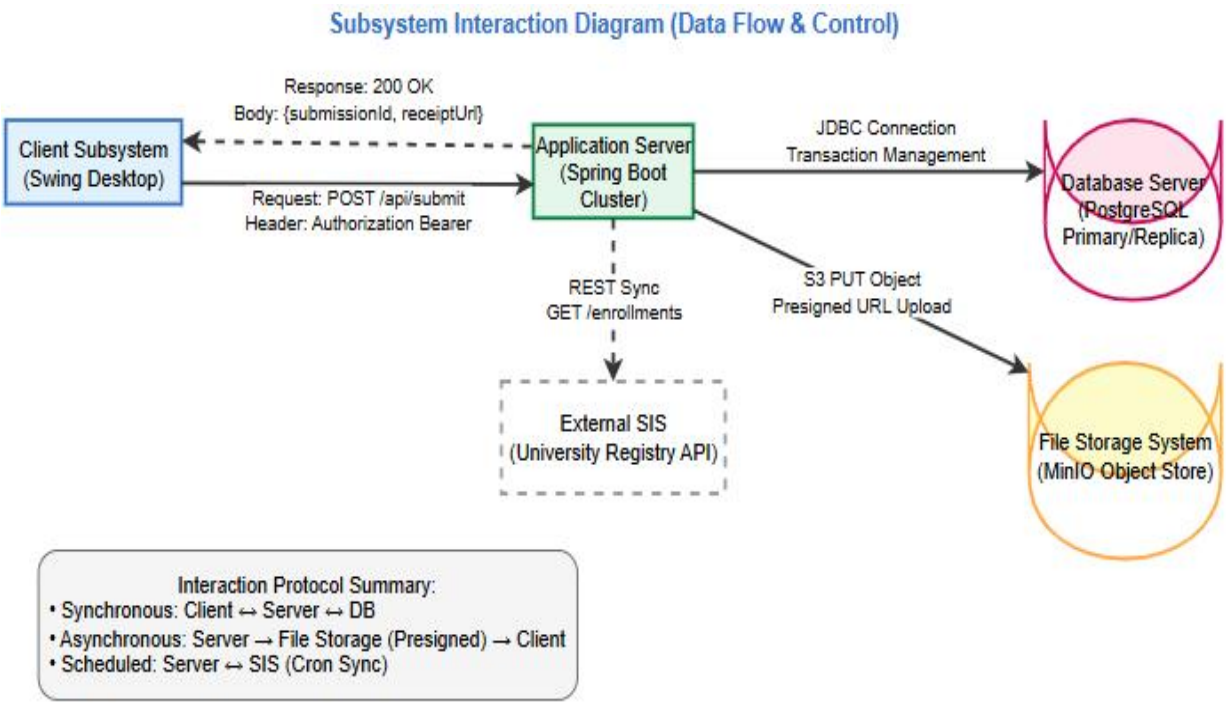


Figure 4.6: Subsystem Interaction Diagram

4.5 Allocation View

The allocation view maps logical software components to physical and virtual hardware infrastructure.

- **Workstations (Edge Nodes)**: Student/Faculty PCs (Windows 11 / Ubuntu 22.04 LTS) running Java 17 JRE. Allocates SwingClientApp.
- **Application Servers (VM Cluster)**: Two 4-vCPU Ubuntu VMs running Dockerized Spring Boot instances. Allocates APIGateway, BusinessServices, AsyncWorkers.
- **Database Server (Primary/Replica)**: High-memory VM with NVMe SSD. Allocates PostgreSQL 15 with synchronous replication.
- **Storage Node**: Dedicated 10TB NAS running MinIO server. Allocates Object Storage.
- **Network Segmentation**: Clients connect via Campus LAN/WiFi to DMZ (Nginx). App servers reside in Private VLAN. DB/Storage isolated in Secure Storage VLAN. Firewall rules enforce least-privilege access.

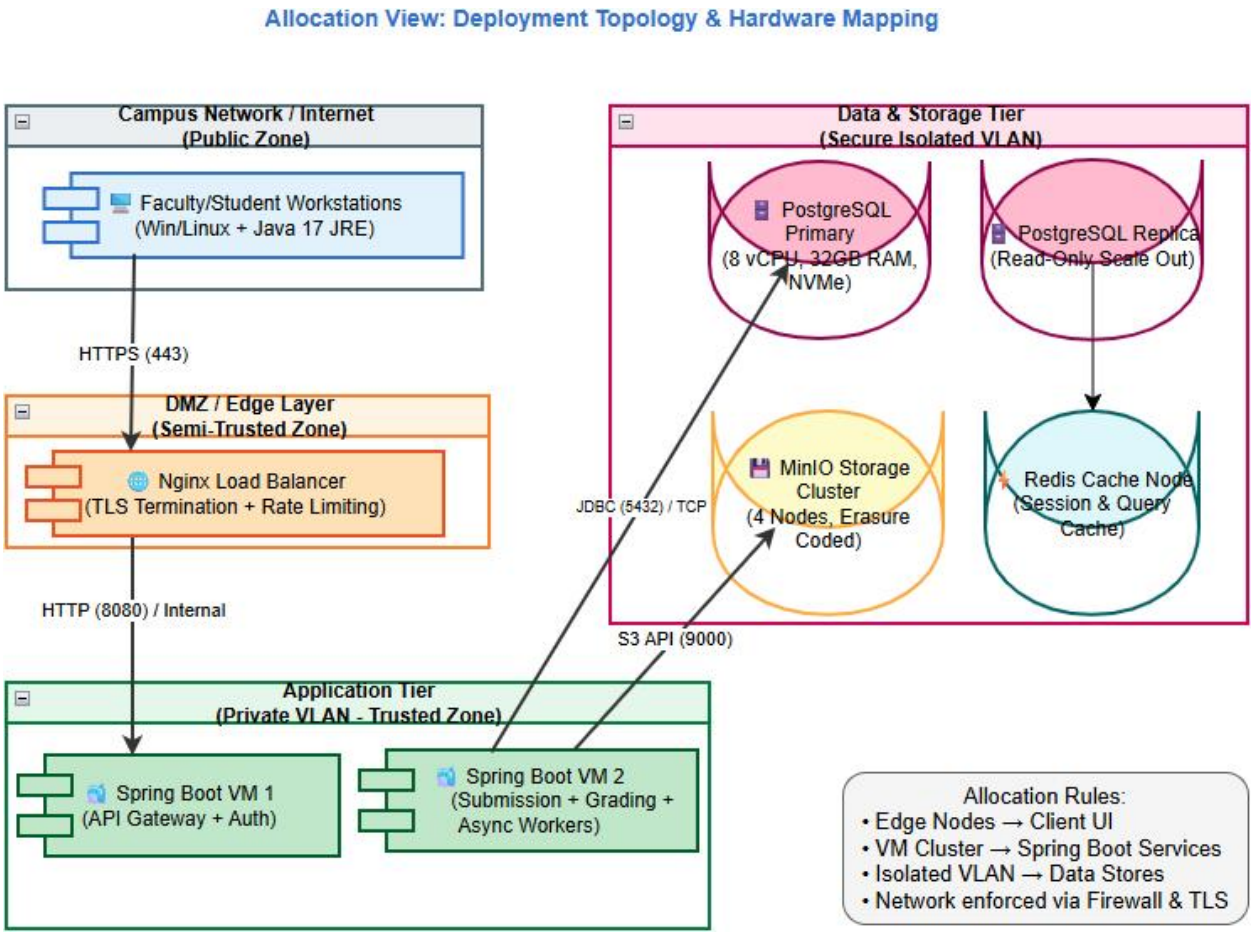


Figure 4.7: Allocation & Deployment View

4.6 Design Rationales

The selected architecture was chosen after evaluating alternatives against team constraints, scalability requirements, and security mandates.

- ★ **Layered vs. Microservices**: Microservices offer independent scaling but introduce operational complexity (service mesh, distributed tracing, eventual consistency) unsuitable for a 4-person team with a 5-month deadline. The layered architecture provides clear boundaries, simplifies debugging, and enables straightforward containerization. Future extraction of high-load services (File Processing) is planned if metrics dictate.
- ★ **Client-Server vs. Pure Web (React/Angular)**: A Java Swing desktop client was mandated to align with university lab infrastructure and provide robust offline draft

capabilities. Client-server architecture ensures centralized data security while leveraging native OS file system access for large project uploads.

★ **Monolithic Deployment vs. Serverless**: Serverless introduces vendor lock-in and cold-start latency. Monolithic Spring Boot deployment ensures predictable performance, easier transaction management, and lower infrastructure costs on university VMs.

★ **Trade-offs**: Network dependency for the Swing client is mitigated via connection pooling, request caching, and offline draft storage. Tight coupling between layers is reduced through interface abstraction (ISubmissionService) and dependency injection, preserving modifiability without sacrificing simplicity.

Chapter 5

DATA DESIGN

5.1 Data Description

The information domain is transformed into a normalized relational schema optimized for transactional integrity and analytical querying. Academic entities are stored in PostgreSQL with strict foreign key constraints to enforce referential consistency. Large binary payloads (assignments, project files, feedback attachments) are externalized to MinIO object storage; only metadata, cryptographic hashes, and secure access tokens persist in the relational database. Redis caches session states, active deadline windows, and frequently accessed course rosters to reduce database load. Data flows from validated client requests through service-layer transactions to persistent stores, with all state mutations logged for audit compliance and rollback capability.

5.2 Data Dictionary

Alphabetical Entity Listing:

Table 5.1: Data Dictionary

Entity	Attributes & Types	Description
Assignment	id (UUID), course_id (UUID), title (VARCHAR), description (TEXT), deadline (TIMESTAMP), max_score (INT), rubric_id (UUID)	Defines submission parameters, due dates, and evaluation criteria
Course	id (UUID), code (VARCHAR), name (VARCHAR), instructor_id (UUID), semester (VARCHAR)	Academic offering container linking faculty, students, and curriculum
Grade	id (UUID), submission_id (UUID),instructor_id (UUID), score (DECIMAL),feedback (TEXT), graded_at (TIMESTAMP)	Evaluated result with instructor commentary and timestamp
Notification	id (UUID), user_id (UUID), type (VARCHAR), message (TEXT), is_read (BOOL),created_at (TIMESTAMP)	System alerts tracking deadline, submission, and grading events
Submission	id (UUID), assignment_id (UUID), student_id (UUID), file_url (TEXT),submitted_at (TIMESTAMP), status (VARCHAR)	Student upload record with lifecycle tracking
User	id (UUID), email (VARCHAR), password_hash (VARCHAR), role (VARCHAR), full_name (VARCHAR),department (VARCHAR)	Authenticated participant profile with institutional role assignment

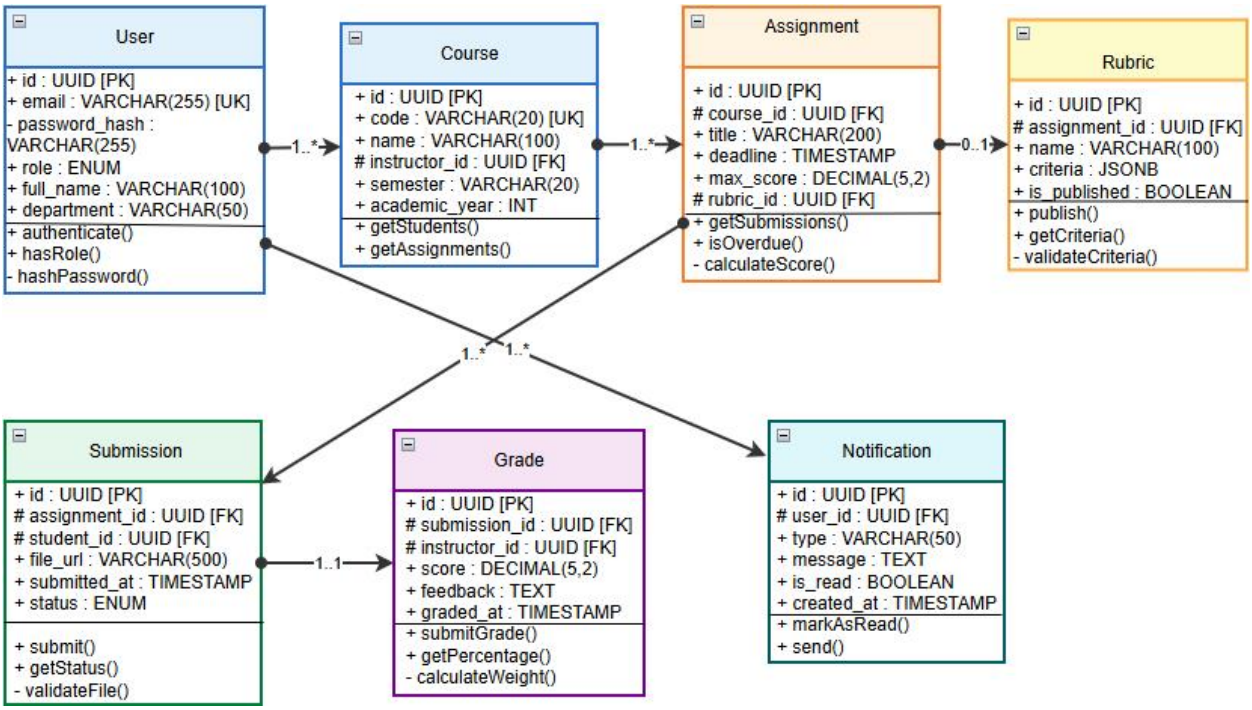


Figure 5.1: Database Schema and Entity-Relationship Diagram (ERD)

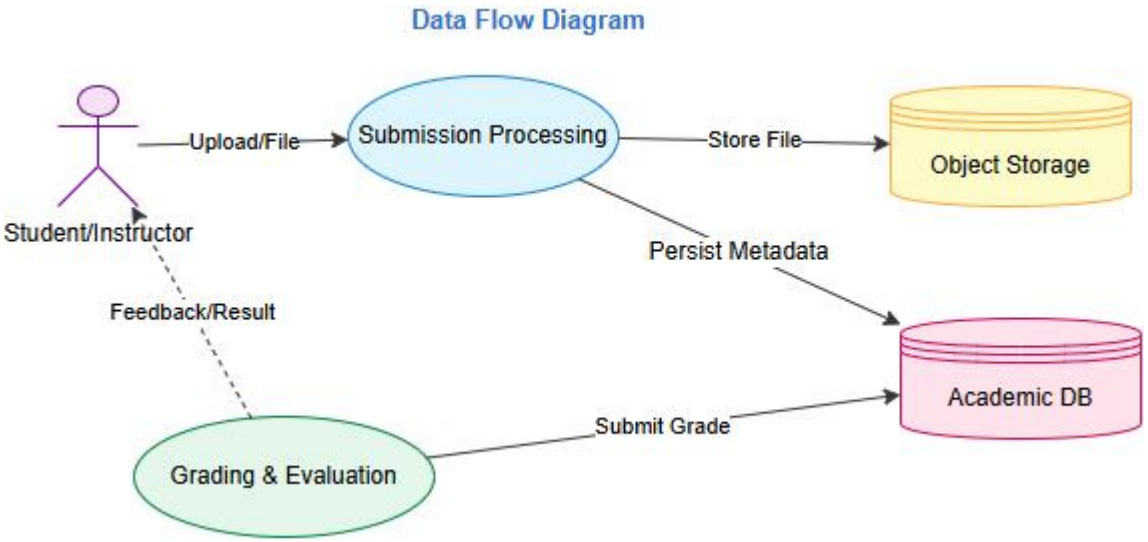


Figure 5.2: Data Flow Diagram (Level 0)

Chapter 6

COMPONENT DESIGN

6.1 Use Case Diagram & Pseudocode

In this section, we detail component behavior. Given the Object-Oriented description in Section 4.2, the following pseudocode summarizes core object member functions for the SubmissionService and GradingService.

```
// SubmissionService.java
FUNCTION submitAssignment(studentId, assignmentId, fileStream):
    TRY
        validateUserRole(studentId, "STUDENT")
        assignment = assignmentRepository.findById(assignmentId)
        IF assignment.deadline < CURRENT_TIMESTAMP AND NOT assignment.allowLate THEN
            THROW BusinessRuleException("Submission deadline has expired")
        ENDIF

        fileMetadata = objectStorage.uploadChunked(fileStream, maxSize="500MB")
        submission = new Submission(studentId, assignmentId, fileMetadata.url, NOW(), "PENDING")
```

```
            TRANSACTION BEGIN
                submissionRepository.save(submission)
                notificationService.enqueue(assignment.instructorId, "New submission received")
            TRANSACTION COMMIT

            RETURN new SubmissionResponse("SUCCESS", submission.id)
        CATCH Exception e
            TRANSACTION ROLLBACK
            objectStorage.delete(fileMetadata.url)
            AUDIT_LOG.record("Submission failed", e.message)
            RETURN new SubmissionResponse("ERROR", e.message)
END FUNCTION

// GradingService.java
FUNCTION calculateGrade(submissionId, rubricScores, instructorId):
        TRY
            submission = submissionRepository.findById(submissionId)
            IF submission.status == "GRADED" THEN
                THROW BusinessRuleException("Submission already graded")
            ENDIF

            totalScore = 0.0
            FOR EACH criterion IN submission.assignment.rubric.criteria DO
                totalScore += rubricScores[criterion.id] * criterion.weight
            END DO

            grade = new Grade(submissionId, instructorId, totalScore, NOW())
            gradeRepository.save(grade)

            submission.status = "GRADED"
            submissionRepository.update(submission)

            notificationService.enqueue(submission.studentId, "Your assignment has been graded")
            RETURN new GradeResponse("SUCCESS", totalScore)
```

```
    CATCH Exception e
        AUDIT_LOG.record("Grading failed", e.message)
        RETURN new GradeResponse("ERROR", e.message)
END FUNCTION
```

Use case Diagram

The system supports three primary actor roles. Students submit work and retrieve evaluations. Instructors configure assignments, apply rubrics, and publish grades. Administrators manage system configuration, synchronize external data, and generate institutional reports.

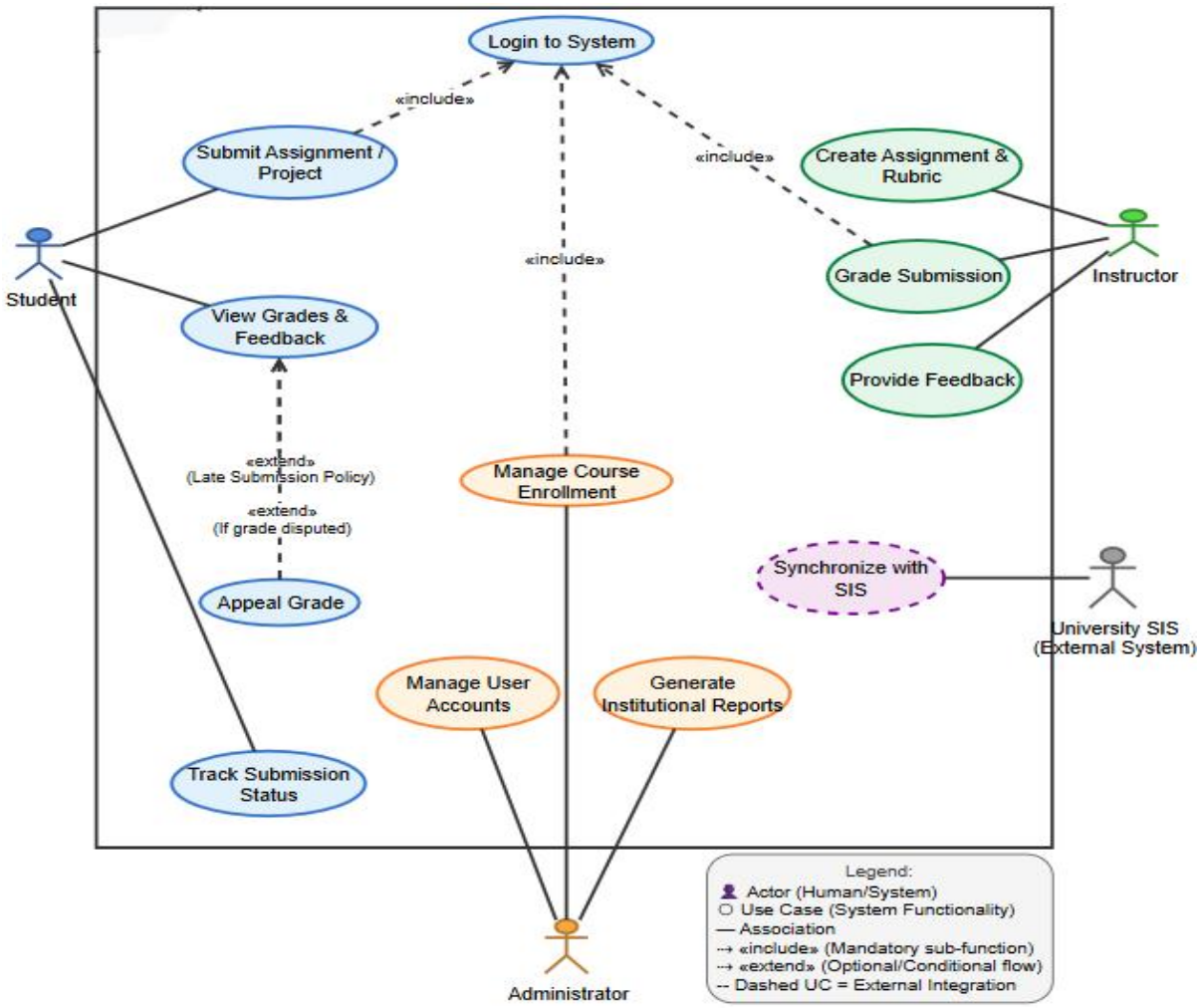


Figure 6.1: UML Use Case Diagram

Use Case Descriptions (Detail Major Use Cases):

1. **Submit Assignment:** Student logs in, navigates to assignment list, clicks upload, selects file via JFileChooser, system validates size/format, uploads to server, displays confirmation dialog.
2. **Grade Submission**: Instructor opens grading panel, system fetches ungraded submissions, instructor opens file in embedded viewer, applies rubric scores in swing table, clicks submit, system calculates total, persists grade, triggers notification.
3. **Create Assignment & Rubric**: Instructor defines title, deadline, max score, adds criteria rows with weights, saves. System validates weights sum to 100%, stores configuration, broadcasts to enrolled students.

6.2 Deployment Diagram

The deployment view illustrates the physical allocation of software artifacts to hardware nodes, defining the topology of the runtime system. This view addresses the Allocation Structure, specifically showing how the Java Swing desktop client, Spring Boot backend services, database clusters, and file storage systems are distributed across the university's physical and virtual infrastructure.

Physical Architecture Description

The system is deployed across four primary hardware zones:

- ★ **Edge/Client Nodes**: Faculty and student workstations running the Java Swing Client. These nodes execute the presentation logic and handle local file selection via the OS file system.
- ★ **DMZ (Demilitarized Zone)**: Hosts the Nginx Load Balancer acting as the reverse proxy. It terminates TLS connections and routes traffic to the application tier.
- ★ **Application Tier**: Consists of a cluster of Virtual Machines running Dockerized Spring Boot Containers. This tier hosts the APIGateway, SubmissionService, GradingService, and AuthService.
- ★ **Data & Storage Tier**: Segregated from the application tier for security. Includes the PostgreSQL Database Cluster (Primary/Replica), Redis Cache Nodes, and MinIO Object Storage Nodes for large file persistence.

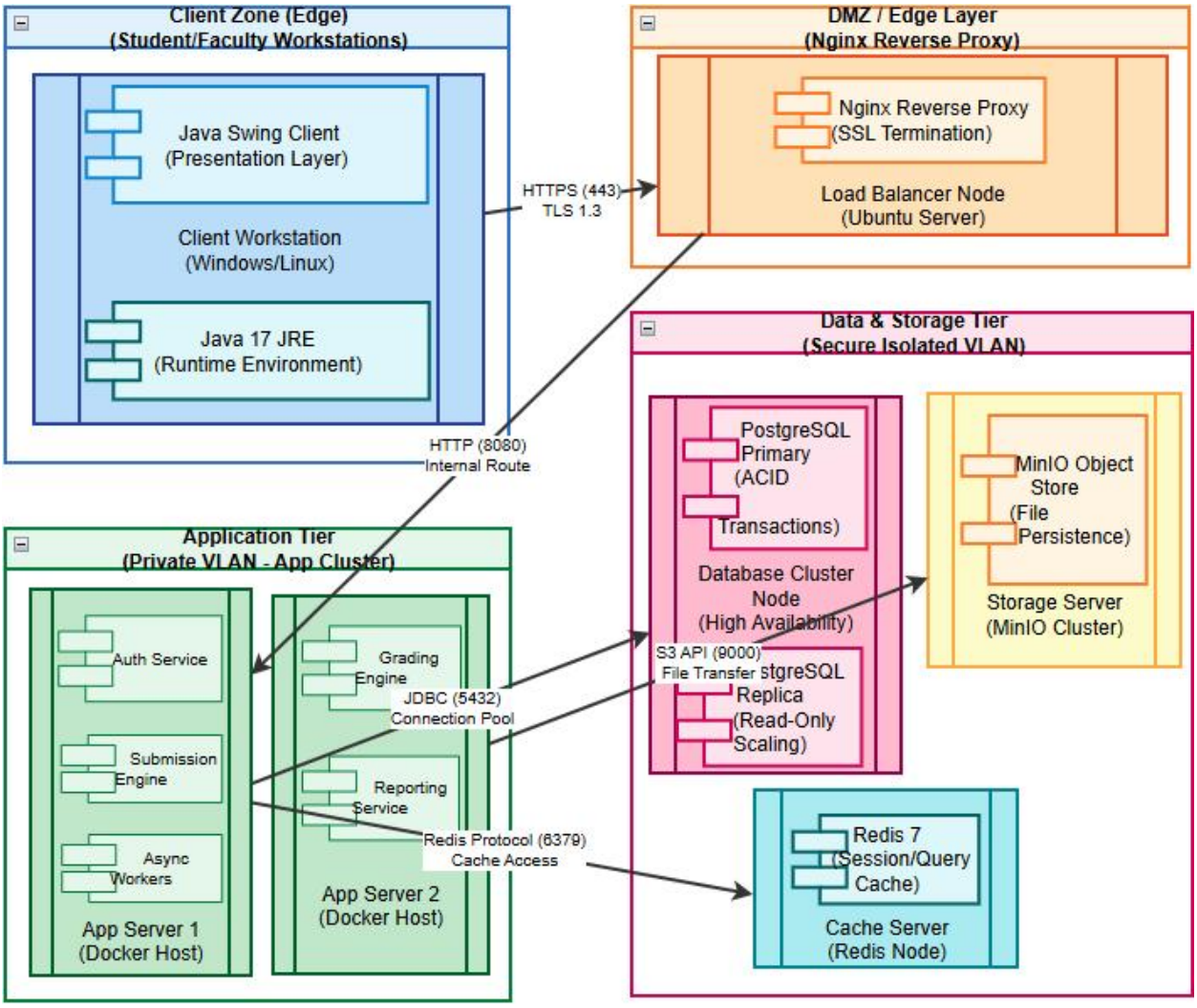


Figure 6.2: Deployment Diagram

6.3 Environment Description

The system operates within the existing IT infrastructure of Mekdela Amba University, utilizing specific hardware and software configurations to ensure security, performance, and compatibility.

Hardware Requirements

- **Client Workstations:**
  - **Architecture**: x86-64 (Intel/AMD)
  - **RAM:** Minimum 4GB (8GB recommended for heavy project file handling).
  - **Storage:** 5GB free space for local caching of uploads and JRE.

- **Server Infrastructure (Virtualized):**
    - **Application Cluster**: 2x Virtual Machines (4 vCPU, 8GB RAM) running Docker containers.
    - **Database Server**: 1x Virtual Machine (8 vCPU, 16GB RAM, 500GB NVMe SSD) configured with RAID 1 for redundancy.
    - **Storage Node**: Dedicated 4TB storage volume accessible via the internal network for MinIO.

Software Requirements

- **Operating Systems:**
    - **Clients**: Windows 10/11 (64-bit) or Ubuntu Linux 20.04 LTS or later.
    - **Servers**: Ubuntu Server 22.04 LTS (LTS for long-term support).
- **Runtime & Middleware:**
    - **Client Runtime**: Java SE 17 JRE (Java Runtime Environment).
    - **Server Runtime**: Java SE 17 JDK (Java Development Kit).
    - **Reverse Proxy**: Nginx 1.24+.
    - **Container Engine**: Docker 24.0+ and Docker Compose.
    - **Database**: PostgreSQL 15, Redis 7.
    - **Object Storage**: MinIO Server (S3 Compatible).

Network Configuration

- **VLAN Segmentation**: The architecture enforces network isolation. The Client Zone communicates with the DMZ via HTTPS (Port 443). The DMZ communicates with the Application Tier via HTTP (Port 8080). The Application Tier communicates with the Data Tier via internal JDBC and Redis protocols, which are blocked from external access by the university firewall.
- **Protocols:**
    - **HTTPS/TLS 1.3**: For all external client communication.
    - **JDBC**: For database connectivity.
    - **SMTP**: For outbound email notifications.
    - **S3 API**: For internal file storage communication.

Chapter 7

REQUIREMENTS MATRIX

The following table traces the system components and data structures back to the Functional Requirements defined in the Software Requirements Specification (SRS).

Table 7.1: Requirements Traceability Matrix

SRS Ref	Requirement Description	Satisfying Component(s)	Data Structure(s)
FR-01	**User Authentication & RBAC**: System must authenticate users via university credentials and assign roles (Student, Instructor, Admin).	AuthService, SecurityConfig, SwingLoginFrame	User, Session, JWT
FR-02	**Course Management:** Instructors must be able to create courses and assign students.	CourseService, CourseController, AdminPanel	Course, Enrollment
FR-03	**Assignment Creation:** Instructors must define assignment parameters (title, deadline, max score) and attach grading rubrics.	AssignmentService, RubricBuilderDialog	Assignment, Rubric, RubricCriterion
FR-04	**File Submission**: Students must be able to upload assignment files securely with format validation and size limits.	SubmissionService, FileUploaderDialog, FileValidator	Submission, FileMetadata
FR-05	**Submission Tracking**: The system must track submission timestamps and enforce late submission penalties based on configuration.	SubmissionService, DeadlineScheduler	Submission, LatePolicy
FR-06	**Grading & Feedback**: Instructors must be able to evaluate submissions using rubrics and provide text feedback.	GradingService, GradingPanel, PDFViewer	Grade, GradeDetail, Feedback
FR-07	**Notification System:** The system must notify users of new assignments, deadlines, and grading results via email and in-app alerts.	NotificationService, EmailWorker	Notification, EmailQueue
FR-08	**Reporting**: Administrators must generate reports on submission rates, grade distributions, and course performance.	ReportService, ReportGenerator, JasperReports	ReportData, CourseStats
FR-09	**SIS Integration:** The system must synchronize course enrollment data with the University Student Information System.	SISIntegrationService, SISAdapter	EnrollmentMapping ,SISUser
FR-10	**Audit Logging**: All critical actions (login, submission, grading) must be logged for security auditing.	AuditService, AuditLogger	AuditLog, EventRecord

DESIGN DECISIONS

The following table records critical architectural and design decisions, including the alternatives considered, the trade-offs analyzed, and the justification for the selected approach.

Table 7.2: Design Decisions

Design Decision	Alternatives Considered	Trade-offs	Justification
Architecture Style	Layered Monolith vs. Microservices	**Trade-off**: Microservices enable independent scaling but add operational complexity and infrastructure overhead. Layered monoliths simplify deployment/debugging but couple module releases.	**Selected**: Layered Monolith. Fits the 4-person team and 5-month timeline. Maintains separation of concerns with minimal overhead; high-load services can be extracted later if needed.
Client Technology	Java Swing Desktop vs. Web SPA (React/Angular)	**Trade-off**: Web SPAs offer cross-platform access but struggle with large uploads and require constant connectivity. Swing enables native file handling but needs local JRE installation.	**Selected**: Java Swing Desktop. Aligns with university Java labs. Native OS integration prevents browser bottlenecks for large project archives over congested campus networks.
Database Engine	PostgreSQL (Relational) vs. MongoDB (NoSQL)	**Trade-off**: MongoDB allows flexible schemas but weakens transactional integrity and reporting joins. PostgreSQL enforces ACID compliance but requires schema migrations.	**Selected**: PostgreSQL. Academic grading demands strict data consistency. Relational models match fixed course/grade entities, simplifying complex institutional reporting.
File Storage	Local Server Disk vs. Object Storage (MinIO)	**Trade-off**: Local disks are simple but create single points of failure and scale poorly. Object storage offers durability and scalability but increases configuration complexity.	**Selected**: MinIO Object Storage. Offloads heavy binaries from the DB.Enables parallel uploads, improves system performance, and simplifies backup strategies.

Authentication	Stateful Session (Cookies) vs. Stateless (JWT)	**Trade-off:** Stateful sessions are simpler to implement but hard to scale. JWTs are stateless and scalable but complicate token revocation.	**Selected**: Stateless JWT. Supports stateless API Gateway routing. Reduces DB lookup latency; revocation is efficiently managed via a Redisblacklist.
Async Processing	Synchronous Threads vs. Message Queue (RabbitMQ)	**Trade-off**: Synchronous calls are easy to debug but block threads during heavy operations. Message queues decouple tasks but require broker maintenance.	**Selected**: Message Queue (RabbitMQ/Spring Events). Prevents blocking the Swing EDT during file processing and email dispatch,ensuring UI responsiveness and immediate user feedback.

.